

Morphic Studio Software Architecture Specification

Document 46 — Testing, Quality Assurance & Reliability Engineering Specification

Version: 1.0

Status: Approved

1. Purpose

This document defines the testing, quality assurance, reliability engineering, and release validation strategy for Morphic Studio.

Its objective is to ensure that every component of the platform—including software, AI workflows, infrastructure, and user experience—meets consistent standards for correctness, performance, accessibility, security, and reliability before release.

2. Quality Philosophy

Quality follows these principles:

2.1 Quality by Design

Testing begins during architecture and implementation rather than after development.

Every feature should be designed to be testable.

2.2 Continuous Verification

Every code change automatically triggers validation.

Quality is continuously measured throughout development.

2.3 Automation First

Manual testing supplements automation but does not replace it.

Automated validation should cover the majority of release requirements.

2.4 AI Evaluation as a Core Discipline

AI capabilities require ongoing evaluation in addition to traditional software testing.

Prompt changes, model upgrades, and provider replacements must be benchmarked before release.

2.5 Reliability Over Feature Count

Releasing fewer reliable features is preferable to releasing more unstable ones.

3. Testing Pyramid

Testing should emphasize:

- Unit Tests
- Integration Tests
- API Tests
- Component Tests
- End-to-End Tests

The majority of tests should be unit and integration tests for fast feedback.

4. Unit Testing

Every business capability should include unit tests for:

- Business rules
- Validation
- Utilities
- Services
- Domain logic

Tests should be deterministic and independent.

5. Integration Testing

Validate interactions between:

- Frontend and backend
- APIs and databases
- Services
- AI Gateway
- Background workers
- Search
- Object storage

Integration tests verify that components function together correctly.

6. End-to-End Testing

Critical user workflows should be automated.

Examples include:

- Account creation
- Project creation
- Story generation
- Character management
- Asset upload
- AI-assisted editing
- Rendering
- Exporting
- Publishing

These tests simulate real user behavior.

7. Accessibility Testing

Verify compliance with WCAG 2.2 AA.

Automated and manual testing should cover:

- Keyboard navigation
- Screen readers
- Focus management
- Contrast

- Form accessibility
- Reduced motion

Accessibility defects are treated as functional defects.

8. Performance Testing

Measure:

- Page load times
- API latency
- Database performance
- Queue throughput
- AI response times
- Rendering performance
- Search speed

Performance targets should be documented and monitored.

9. Load & Stress Testing

Evaluate platform behavior under increasing demand.

Scenarios include:

- Concurrent users
- AI generation spikes
- Large asset uploads
- Bulk exports
- High-frequency API requests

The platform should fail gracefully under extreme load.

10. AI Evaluation Testing

Maintain benchmark datasets for AI capabilities.

Evaluate:

- Output quality
- Consistency

- Hallucination rate
- Safety
- Latency
- Cost
- Canonical Story Bible adherence

AI regression testing should detect unintended changes caused by prompt updates or provider changes.

11. Visual Regression Testing

Automatically compare user interfaces across releases.

Detect unintended changes in:

- Layout
 - Typography
 - Components
 - Themes
 - Responsive behavior
-

12. Security Testing

Perform:

- Dependency scanning
- Static application security testing
- Dynamic security testing
- Authentication testing
- Authorization testing
- API security testing
- Secret detection

Critical vulnerabilities block releases until resolved.

13. Reliability Engineering

Define Service Level Objectives (SLOs) for critical services.

Examples include:

- API availability

- AI Gateway availability
- Search availability
- Export success rate
- Render success rate

Track error budgets and reliability trends over time.

14. Incident Management

Document procedures for:

- Detection
- Triage
- Mitigation
- Recovery
- Root cause analysis
- Post-incident review

Every significant incident should produce documented lessons learned.

15. Release Quality Gates

A release should not proceed unless:

- All automated tests pass.
 - Critical defects are resolved.
 - Security scans pass.
 - Accessibility requirements are met.
 - Performance targets are satisfied.
 - AI benchmark scores remain within acceptable thresholds.
 - Deployment verification succeeds.
-

16. Continuous Improvement

Quality metrics should be reviewed regularly.

Examples include:

- Test coverage
- Defect trends
- AI evaluation scores

- Deployment success rate
- Incident frequency
- Mean time to recovery (MTTR)
- Customer-reported issues

These metrics guide ongoing improvements.

17. Acceptance Criteria

The testing architecture is successful if:

- Every major feature is testable.
 - Automated testing provides rapid feedback.
 - AI behavior is continuously evaluated.
 - Accessibility and security are verified.
 - Releases are consistently reliable.
 - Operational reliability improves over time.
 - Quality remains measurable as the platform evolves.
-

Conclusion

The Morphe Studio testing and reliability architecture establishes a comprehensive quality framework that extends beyond traditional software testing to include AI evaluation, accessibility, security, performance, and operational excellence. By treating quality as a continuous engineering discipline, the platform is positioned to deliver dependable creative experiences while evolving safely and confidently.